

Project Recommendation

Cameron Moses | Frontend Software Engineer | May 2026

Your recommended project: Client Intake and Case Progress Tracker

A web application for real estate professionals to manage client pipelines, track deal stages, and centralise communication - built from your direct operational experience as a Realtor and Real Estate Salesperson.

Why This Project

Your background data makes the recommendation straightforward. You have worked as a licensed Realtor with eXp Realty in San Diego and as a Real Estate Salesperson at JJJ Properties. That is a combined two-plus years of active experience operating inside a client management workflow - handling intake, tracking deal stage progression, managing document checklists, and maintaining ongoing client communication.

That is not adjacent to a problem. That is the problem. You have sat inside the workflow this tool is designed to fix. You know which parts of it are manual, which are done in spreadsheets, which require chasing clients for documents, and where deals stall because of communication gaps. No amount of market research produces that kind of product insight.

The project is not chosen because real estate is a trendy vertical or because CRM tools are popular. It is chosen because you can explain the specific operational frustration that motivated it, name the exact moment in the client lifecycle where it would have saved you time, and describe a real user - yourself, six months ago - with complete credibility. That is exactly what hiring managers are listening for when they ask: why did you build this?

This project also directly closes every skill gap identified in your review: Jest and React Testing Library for the component suite, Node.js and Express for the API layer, and GitHub Actions for CI. None of these are bolted on - they are the natural implementation path for the features described below.

The Problem

Real estate agents and small brokerages manage multiple active client relationships simultaneously. A typical pipeline includes leads at various stages: initial inquiry, pre-qualification, property search, offer submitted, under contract, pending close, and closed. The problem is that the tools most agents actually use are not purpose-built for this workflow.

Many agents manage their pipeline in a spreadsheet. Some use a general CRM that was not designed for residential real estate and requires significant configuration. Others use paper and email. The result is predictable: deals fall through gaps, follow-up tasks get missed, document checklists are tracked inconsistently, and the agent has no single view of where each client stands today.

The specific pain you experienced - and that this application solves - is the absence of a lightweight, purpose-built tool that gives a solo agent or small team: one clear view of every active client, their current deal stage, outstanding tasks, and the next action required. Not a bloated enterprise CRM. Not a spreadsheet. A focused, fast, field-relevant tool.

The Project

You are building a Client Intake and Case Progress Tracker for real estate professionals. It is a single-page React application with a Node.js API backend and a Firebase Firestore database. Here is what it does from end to end.

The dashboard view is the home screen. It shows every active client as a card. Each card displays the client name, current deal stage (one of seven stages from the pipeline above), the most recent note, and a colour-coded urgency indicator based on how many days since the last activity. Deals with no activity in over five days are flagged in amber. Over ten days, red.

The client detail view opens when you click any card. It shows the full client profile: contact information, property address or search criteria, current stage, a checklist of required documents by stage, a chronological activity log, and a notes field. Each checklist item can be marked complete. The activity log updates automatically when a note is added or a stage changes.

The stage management feature lets the agent drag a client card from one stage column to another - like a Kanban board - or update the stage from a dropdown in the detail view. Stage changes are logged automatically with a timestamp.

The add client form handles intake: name, contact details, property type, budget range, and an optional first note. Validation is handled with React Hook Form, which you already have in your stack.

Secondary features: a simple search and filter bar on the dashboard (filter by stage, search by name), and an export function that generates a plain-text summary of any client record - useful for agents who need to share status updates with a colleague or broker.

Why It Works for Your Portfolio

A hiring manager reviewing your portfolio will see three things in this project that most bootcamp graduate portfolios do not have.

First, a domain problem that is specific and real. You can say: I worked as a Realtor and I built this because I was managing my pipeline in a spreadsheet. That sentence does more work than any technical specification. It tells the hiring manager you think about users before you think about code.

Second, a technical implementation that is not a tutorial clone. This is an original architecture with a real API, a real database schema, real form validation, and real state management across multiple views. It is not a Netflix interface with a different colour scheme.

Third, it demonstrates the exact skills that most frontend job descriptions flag as either required or highly preferred: TypeScript throughout, React with complex component composition, a tested component suite, a Node.js backend, and a CI workflow. This project alone can close four of the five skill gaps identified in your review.

Technology Stack

- React 18: your primary framework - component composition, hooks, and context are all exercised by this project's dashboard-to-detail navigation pattern.
- TypeScript: already in your stack via the E-Commerce project. Using it here across both the frontend components and the Express API models reinforces your proficiency with a consistently job-listed requirement.
- Next.js (App Router): chosen over plain React because you have production experience with it via Skinstric AI. Using it here with API routes means you can keep the project as a single repository with a unified deployment.
- Tailwind CSS: already your preferred styling approach. Chosen over alternatives because your existing component patterns transfer directly.
- React Hook Form: already in your stack. Used for the client intake form with Zod schema validation added - Zod is a small addition that demonstrates type-safe form handling, increasingly seen in frontend job specs.
- Node.js / Express (via Next.js API routes): the API layer. Next.js API routes give you Node.js experience without requiring a separate server deployment - lower operational complexity while still evidencing the skill.

- Firebase Firestore: already in your stack from the Netflix Clone. Chosen over PostgreSQL or MongoDB for this project because the real-time listener capability enables the activity log and urgency indicators to update without polling.
- Jest and React Testing Library: the primary skill gap this project closes. Every UI component in the dashboard and detail views is a natural test target. You are aiming for at least 10 component tests before the project is portfolio-ready.
- GitHub Actions: a CI workflow that runs your Jest suite on every push. Takes roughly 30 minutes to configure once the tests exist. Adds a green badge to the README.
- Vercel: your existing deployment platform. No change needed.

How It Closes Your Skill Gaps

Jest and React Testing Library

The ClientCard component - the individual card on the dashboard - is the perfect first test target. You can write tests for: renders the correct client name, shows amber urgency indicator when last activity was 6 days ago, shows red urgency indicator when last activity was 11 days ago, and calls the onClick handler when clicked. That is four meaningful tests on a single component. The StageDropdown component and the AddClientForm are the next two targets. By the end of the project you will have a test suite that covers the core user journeys.

Node.js / Express

The Next.js API routes give you the Node.js experience you need without the overhead of a separate server. You will write routes for: GET /api/clients (list all clients), POST /api/clients (create a new client), PUT /api/clients/:id (update stage or add a note), and GET /api/clients/:id (get a single client detail). Each route is a small, focused function. By the end you will be able to describe REST API design, request validation with Zod, and error handling with confidence.

GitHub Actions

Once your Jest suite exists, the CI workflow is a single YAML file. The workflow runs on every push to main, installs dependencies, and runs npm test. If any test fails, the push is flagged. The green badge in the README is visible evidence to every recruiter who clicks your GitHub link.

Accessibility (WCAG basics)

The stage Kanban board and the client detail view both require keyboard navigation and screen-reader-compatible markup. Build these with semantic HTML from the start - use button elements for interactive items, label all form inputs, and ensure colour is not the only indicator of urgency status. This is a natural implementation requirement, not an add-on.

Week-by-Week Build Plan

Based on approximately 10 hours per week. Adjust if your schedule allows more.

Week 1: Setup and Data Modelling

- Initialise Next.js project with TypeScript and Tailwind CSS
- Define TypeScript interfaces: Client, DealStage, ActivityLog, DocumentChecklist
- Set up Firebase Firestore with security rules
- Create the database schema: clients collection with sub-collections for activity and documents
- Write the first two API routes: GET /api/clients and POST /api/clients
- Deliverable: project runs locally, two working API routes, data model documented in README

Week 2: Dashboard and Client Cards

- Build the dashboard layout with Tailwind CSS grid
- Build the ClientCard component with urgency colour logic
- Connect dashboard to the GET /api/clients API route
- Write the first Jest tests for ClientCard - urgency indicator logic is the target
- Deliverable: working dashboard that reads from Firestore, first passing test suite

Week 3: Client Detail View

- Build the client detail view: contact info, current stage, document checklist, activity log
- Implement the PUT /api/clients/:id route for stage updates and note additions
- Activity log auto-updates on stage change and note add
- Document checklist items toggle complete/incomplete and persist to Firestore
- Deliverable: full client detail view with working activity log

Week 4: Intake Form and Validation

- Build the AddClientForm using React Hook Form and Zod schema validation
- Wire to POST /api/clients route
- Write tests for the form: required field validation, successful submission, error states
- Add search and filter bar to dashboard
- Deliverable: complete intake flow, form tests passing

Week 5: Kanban Stage View (optional but recommended)

- Build a Kanban-style board view as an alternative to the card grid dashboard
- Implement drag-and-drop stage updates using the HTML Drag and Drop API (no library needed)
- Stage changes persist to Firestore and add to the activity log
- Deliverable: working Kanban view with drag-and-drop stage management

Week 6: Polish, README, CI, and Deployment

- Set up GitHub Actions workflow - runs Jest on every push
- Fix any failing tests, add CI badge to README
- Write full README: problem statement, screenshots, tech stack, local setup instructions
- Accessibility audit: keyboard navigation, semantic HTML, ARIA labels where needed
- Deploy to Vercel, confirm all routes and real-time features work in production
- Deliverable: live deployment, green CI badge, complete README - portfolio-ready

Minimum Viable Product

The following features must be complete before this project is portfolio-ready. Done means: the feature works in the live deployment, it is covered by at least one test, and it is documented in the README.

- Dashboard showing all active clients as cards with urgency indicators based on last activity date.
- Client detail view with contact info, current deal stage, document checklist (with persistent completion toggle), and chronological activity log.
- Add client form with validation - required fields enforce on submit, form resets on success.
- Stage updates via dropdown or Kanban drag - changes persist to Firestore and appear in the activity log.
- At least 10 passing Jest and React Testing Library tests covering the ClientCard, AddClientForm, and StageDropdown components.
- GitHub Actions CI workflow running on every push with a green badge in the README.
- Live Vercel deployment with all features functional in production.
- README with problem statement, screenshots, tech stack, and local setup instructions.

Stretch Features

- Email reminders via Resend or SendGrid: send an automated email to the agent when a client has had no activity for 7 days. Demonstrates API integration and server-side scheduling - both listed skills in mid-level frontend JDs.
- Document upload: allow clients to upload required documents (ID, bank statements, etc.) directly to Firebase Storage. Demonstrates file handling and storage integration.
- Closing timeline visualisation: a simple timeline component that shows the expected key dates for a transaction (offer, inspection, appraisal, close). Built with a custom SVG or a lightweight chart library. Demonstrates data visualisation skills.
- Role-based access: a simple two-role model (agent and admin) with different dashboard views. Demonstrates authentication patterns beyond basic Firebase auth.
- Exported PDF summary: generate a one-page PDF summary of any client record using a library such as jsPDF. Directly applicable to the real-world use case of sharing client status with a broker.

README and Portfolio Presentation

Your README should open with the problem, not the technology. Write it like this: I worked as a licensed Realtor managing multiple active client pipelines simultaneously. The standard workflow - tracking deal stages, document checklists, and follow-up tasks - required either an over-engineered enterprise CRM or a spreadsheet. I built this tool as a purpose-built alternative for solo agents and small teams. Then list the features, the tech stack, and the setup instructions.

In your portfolio, present this project with a screenshot of the dashboard in its populated state - not the empty state. Show real-looking (but fictional) client names, deal stages across multiple columns, and at least one card with an urgency indicator active. This is the difference between a hiring manager clicking through and clicking away.

When a hiring manager asks you about this project in an interview, your answer has three parts. First, describe the real problem: you were a Realtor and you built this because you lived the pain. Second, describe the most technically interesting decision you made - the activity log auto-update using Firestore real-time listeners is a strong candidate. Third, describe what you learned: how testing changed the way you think about component design, or what writing API routes taught you about data modelling. Keep the whole answer under two minutes. End by saying the live demo is in your portfolio and the code is on GitHub.

Do not describe this as a to-do list for Realtors. It is not a to-do list. It is a pipeline management tool with real-time state, a tested component suite, and a domain-specific data model. The framing matters as much as the code.